

Conditional statements

Make comparisons using operators

Comparators

Operators that compare two values and produce boolean values(True/False)

Logical operators

- Operators that connect multiple statements together and perform more complex comparisons
 - and
 - or
 - not

Reference guide: Python operators

You've encountered many Python operators already. Many of them likely feel very familiar to you. After all, there's nothing novel about addition and subtraction in Python. But there are many more operators than the ones used for basic arithmetic! Operators are characters that enact specific arithmetic, logical actions, or processes. Data professionals use operators all the time in their work, and they're a rudimentary part of Python programming, so it's important to learn them. This reading is a guide to the various operators available to you in Python.

Save this course item

You may want to save a copy of this guide for future reference. You can use it as a resource for additional practice or in your future professional projects. To access a downloadable version of this course item, click the following link and select "Use Template."

Reference guide: [Python operators](#)

OR

If you don't have a Google account, you can download the item directly from the following attachment.

Reference guide_ Python operatorsDOCX File

Comparators

In Python, you can use comparison operators to compare values. When a comparison is made, Python returns a Boolean result—True or False. Python uses the following comparators:

Operation	Operator
greater than	>
greater than or equal to	>=
less than	<
less than or equal to	<=
not equal to	!=
equal to	==

Notes:

- The single equals sign (=) is reserved for assignment statements. If you use a single equal sign to make a comparison, the computer will return a **SyntaxError**.
- If you try to compare data types that aren't compatible, like checking if a string is greater than an integer, Python will throw a **TypeError**.

Logical operators

Python also has three logical operators that can be combined with comparators to create more complex statements.

These operators are:

- **and**
 - evaluates to True only if both statements are true
- **or**
 - evaluates to True if one or both of the statements are true
- **not**

- reverses the evaluation
- If the statement evaluates to True, returns False; if the statement evaluates to False, returns True

Examples:

```
x = 3
my_list = [3, 4, 6, 10]
print(x < 3 and x != 0) #False
print(x >= len(my_list) or x == min(my_list)) #True
print(x not in my_list) #False
```

Arithmetic operators

Python is also capable of performing mathematical operations using a set of built-in operators. These arithmetic operators are:

Operation	Operator	Example
Addition	+	[IN] 5 + 2 [OUT] 7
Subtraction	-	[IN] 5 - 2 [OUT] 3
Multiplication	*	[IN] 5 * 2 [OUT] 10
Division	/	[IN] 5 / 2 [OUT] 2.5
Modulo (the remainder of a division)	%	[IN] 5 % 2 [OUT] 1
Exponentiation	**	[IN] 5 ** 2 [OUT] 25
Floor division (the number of times the denominator can fully go into the numerator)	//	[IN] 5 // 2 [OUT] 2

There are many other mathematical operations that can be performed in Python using functions from special libraries, which you'll learn about later. Python uses a core set of operators to make comparisons, perform logical operations, and compute arithmetic operations. These operators can be combined in statements to perform an infinite number of tasks and operations.

Use if, elif, else statements to make decisions

Branching

- The ability of program to alter its execution sequence
- Use if statement based on certain conditions

Modulo

- An operator that returns the remainder when one number is divided by another

Uses of branching

- Bin data based on its value
- Backup files
- Restrict login access

Reference guide: Conditional statements

Conditional statements are an essential part of programming. They allow you to control the flow of information based on certain conditions. In Python, **if**, **elif**, and **else** statements are used to implement conditional statements. Using conditional statements to branch program execution is a core part of coding for most data professionals, so it's important to understand how they work. This reading is a reference guide to conditional statements.

Save this course item

You may want to save a copy of this guide for future reference. You can use it as a resource for additional practice or in your future professional projects. To

access a downloadable version of this course item, click the following link and select "Use Template."

Reference guide: [Conditional statements](#)

OR

If you don't have a Google account, you can download the item directly from the following attachment.

Reference guide_ Conditional statementsDOCX File

Conditionals syntax

In earlier videos, you learned some built-in Python operators that allow you to compare values, and some logical operators that you can use to combine values. You also learned how to use operators in **if-elif-else** blocks.

Note: The following code block is not interactive.

The basic syntax of **if-elif-else** statements in Python is as follows:

```
if condition1:
    # block of code to execute if the condition evaluates to True

elif condition2:
    # block of code to execute if condition1 evaluates to False
    # and condition2 evaluates to True

else:
    # block of code to execute if BOTH condition1 and condition2
    # evaluate to False
```

Here, **condition1** and **condition2** are expressions that evaluate to either True or False. If the condition in the if statement is true, then the block of code that follows is executed. Otherwise, it is skipped.

The **elif** statement stands for "else if," and it is used to specify an alternative condition to check if the first condition is false. You can have any number of **elif** statements in your code. If the preceding condition is false and the **elif** condition is true, then the block of code that follows the **elif** statement is executed.

The **else** statement is used to specify what code to execute if both the if statement and any subsequent **elif** statements are false.

Here is an example that uses all three kinds of statements:

```
x = 8
if x > 5:
    print('x is greater than five')
elif x < 5:
    print('x is less than five')
else:
    print('x is equal to five')
```

Omitting else

Often, you'll find that there is no need to use an **else** statement, because it is superfluous in the logical context of your code. Consider this example:

```
def greater_than_ten(x):
    if x > 10:
        return True
    else:
        return False

print(greater_than_ten(15))
print(greater_than_ten(2))
```

This function will return **True** if **x** is greater than 10, otherwise it will return **False**. Notice that **else** is on its own line, followed by an indented **return** statement in the last line. This is perfectly fine code, and you'll encounter code like this routinely.

But there's another form that you'll routinely encounter that skips this last step. Consider this code, which is equivalent to the function above:

```
def greater_than_ten(x):
    if x > 10:
        return True
    return False
```

```
print(greater_than_ten(15))
print(greater_than_ten(2))
```

In this case, there is no **else** statement, but it doesn't make a difference. The logic is the same. The function will begin execution from the top and move down. When it gets to line 2, it will evaluate whether or not **x** is greater than ten, and if so, it will move to the indented line 3, which is a **return** statement. Once this **return** statement executes, the function exits. It does not continue any further execution.

However, if the condition in line 2 does not evaluate as **True**, the function will not execute line 3, because it is an indented sub-condition of line 2. Instead it will proceed directly to line 4, which says that the function must return **False** and exit. Note also that the final statement, **return False**, is at the same indentation level as the **if** statement.

As you continue to gain skills and experience in programming, you might find that you prefer one way over the other. Use whichever way is best for you, and make sure you're familiar with both ways.

Key takeaways

Some important things to note about conditional statements in Python:

- The **elif** and **else** statements are optional. You can have an **if** statement by itself.
- You can have multiple **elif** statements.
- You can only have one **else** statement, and only at the end of your logic block.
- The conditions must be an expression that evaluates to a Boolean value (True or False).
- Indentation matters! The code associated with each conditional statement must be indented below it. The typical convention for data professionals is to indent four spaces. Indentation mistakes are one of the most common causes of unexpected code behavior.